

🔥 РУКОВОДСТВО ПО ИНТЕГРАЦИИ API (M2M AUTOMATION)

Настоящий документ содержит исчерпывающие сценарии межпрограммного взаимодействия с асинхронным бэкендом панели для автоматизации процессов заведения учетных записей, блокировки операторов и наката DDL-прав.

--- Схема сквозной межпрограммной координации доступов ---

ВНЕШНЯЯ СИСТЕМА АВТОМАТИЗАЦИИ (IDM / IAM / CI-CD)

Шаг 0: POST-запрос на /api/login —> Сбор JSON: {"access_token": "eyJ..."}

МАССИВ ПАКЕТОВ: ИНЖЕКТ ЗАГОЛОВКА AUTHORIZATION
Headers: {"Authorization": "Bearer ey...", "Content-Type": "application/json"}

Разветвление по исполнительным сервисным эндпоинтам

1. Создание роли в СУБД

/api/manage-roles

Режим: CREATE_ONLY.

Выдача одноразовых UUID-паролей на Email сотрудника.

2. Точечный накат прав

/api/manage-privileges

Режим: SINGLE_TABLE.

Снайперский лог: фиксация чистого имени объекта без масок.

3. Блокировка в панели

/api/admin/update-user-status

Флаг: is_active=false.

Root Guard: блокиратор учетки admin.

--- Таблица структуры вызова интеграционных эндпоинтов ---

Маршрут и Метод API	Эталонный JSON Payload запроса	ИБ-Предохранитель
POST /api/manage-roles	{ "target_server": "prod-db-01", "username": "kuznetsov_am", "target_email": "kuzn@co.ru", "action_mode": "CREATE_ONLY" }	Запрещает сброс или перехват чужих паролей в СУБД.
POST /api/manage-privileges	{ "target_server": "prod-db-01", "target_db": "orders_db", "username": "app_reader", "scope": "TABLE", "schema_name": "public", }	Фикс логов: пишет в аудит реальный объект без мусорных масок.

1 / 3

	<pre> "table_name": "clients_cards", "action": "GRANT", "privilege": "SELECT" } </pre>	
POST /api/admin/update-user-status	<pre> { "username": "fired_user", "is_active": false, "status": "NOLOGIN" } </pre>	Жестко блокирует попытки заморозить root-аккаунт admin.

ПРИМЕРЫ СЦЕНАРИЕВ ИНТЕГРАЦИИ (PYTHON / CURL)

Внедрение этих проверенных фрагментов гарантирует корректную передачу авторизационных заголовков.

--- Python-скрипт выдачи GRANT-прав на одну таблицу ---

```

import requests

# Шаг 0. Получение Bearer-токена под служебным аккаунтом IDM
auth_payload = {"username": "idm_service_account", "password":
"VeryStrongPassword2026"}
auth_res = requests.post("https://company.ru", json=auth_payload, timeout=5)
token = auth_res.json().get("access_token")

# Настройка защищенных HTTP-заголовков сессии
headers = {
    "Authorization": f"Bearer {token}",
    "Content-Type": "application/json"
}

# Payload автоматического наката привилегий на таблицу после миграции CI/CD
privilege_payload = {
    "target_server": "prod-db-01",
    "target_db": "orders_db",
    "username": "app_reader",
    "scope": "TABLE",
    "schema_name": "public",
    "table_name": "clients_cards",
    "action": "GRANT",
    "privilege": "SELECT"
}

response = requests.post("https://company.ru", json=privilege_payload,
headers=headers, timeout=10)
print(response.json())

```

--- Bash-команда экстренной блокировки скомпрометированного сотрудника ---

```
# 1. Атомарное извлечение JWT-токена из JSON-ответа авторизации
TOKEN=$(curl -s -X POST "https://company.ru" \
  -H "Content-Type: application/json" \
  -d '{"username": "idm_service_account", "password": "VeryStrongPassword2026"}' \

  | grep -o '"access_token": "[^"]*"' | grep -o ' "[^"]*$' )

# 2. Перевод аккаунта в статус NOLOGIN через служебный REST-шлюз
curl -s -X POST "https://company.ru" \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"username": "fired_user", "is_active": false, "status": "NOLOGIN"}'
```